

PROGRAMACIÓN ORIENTADA A OBJETOS EN PYTHON

Sistema de Empleados de una Empresa

Encapsulamiento · Herencia · Abstracción · Polimorfismo

Guía práctica con ejemplos, clases abstractas y pruebas unitarias (unittest)

Python 3 · ABC · @property · unittest

Tabla de Contenido

1. El problema que vamos a resolver

Imaginemos una empresa que tiene diferentes tipos de empleados:

- Empleado administrativo
- Desarrollador
- Gerente

Todos tienen características comunes:

- nombre
- documento
- salario

Pero cada tipo de empleado calcula su bonificación de una manera diferente. Por ejemplo:

- Administrativo → 10% del salario
- Desarrollador → 15% del salario
- Gerente → 20% del salario

Aquí aparece una excelente oportunidad para utilizar polimorfismo.

2. Estructura del proyecto

Podríamos organizarlo así:

```
empresa/  
├── empleado.py  
├── administrativo.py  
├── desarrollador.py  
├── gerente.py  
├── main.py  
└── test_empleados.py
```

Para comenzar, sin embargo, podemos colocar todo en un solo archivo para que sea más fácil de entender.

3. Clase abstracta Empleado

Primero necesitamos una clase que represente el concepto general de empleado. Pero no queremos crear directamente un objeto Empleado: queremos que sea una clase abstracta.

```
from abc import ABC, abstractmethod

class Empleado(ABC):

    def __init__(self, nombre, documento, salario):
        self.nombre = nombre
        self.documento = documento
        self.salario = salario

    @abstractmethod
    def calcular_bonificacion(self):
        pass

    def mostrar_informacion(self):
        print(f"Nombre: {self.nombre}")
        print(f"Documento: {self.documento}")
        print(f"Salario: ${self.salario:,.0f}")

    def __str__(self):
        return f"{self.nombre} - Documento: {self.documento}"
```

Vamos a analizarlo cuidadosamente.

4. ¿Qué significa ABC?

```
from abc import ABC, abstractmethod
```

ABC significa Abstract Base Class, es decir, Clase Base Abstracta. Después:

```
class Empleado(ABC):
```

Estamos indicando que Empleado será una clase abstracta.

5. El constructor

```
def __init__(self, nombre, documento, salario):
    self.nombre = nombre
    self.documento = documento
    self.salario = salario
```

Aquí estamos creando los atributos del objeto. Por ejemplo: `empleado.nombre`, `empleado.documento`, `empleado.salario`.

Si creamos:

```
desarrollador = Desarrollador(
    "Carlos",
    "123456",
    4000000
)
```

el objeto tendrá `nombre = Carlos`, `documento = 123456`, `salario = 4000000`.

6. Abstracción

Esta parte es importantísima:

```
@abstractmethod
def calcular_bonificacion(self):
    pass
```

Estamos diciendo: todo empleado debe saber calcular su bonificación. Pero la clase `Empleado` no sabe exactamente cómo, por eso dejamos el método abstracto. Cada clase hija tendrá que implementar su propia versión. Esto es abstracción.

La idea conceptual es:

```
Empleado
├── debe calcular bonificación
│   ├── Administrativo → 10%
│   ├── Desarrollador → 15%
│   └── Gerente → 20%
```

7. ¿Por qué usamos pass?

```
def calcular_bonificacion(self):
    pass
```

`pass` significa "no hacer nada". Aquí se utiliza porque estamos definiendo la estructura del método, pero serán las clases hijas las que proporcionarán la implementación.

8. Herencia

Ahora vamos a crear:

```
class Administrativo(Empleado):
```

Esto significa que Administrativo hereda de Empleado. Código:

```
class Administrativo(Empleado):

    def calcular_bonificacion(self):
        return self.salario * 0.10
```

El empleado administrativo automáticamente recibe nombre, documento, salario, mostrar_informacion() y __str__(), porque los hereda de Empleado. Pero además tiene calcular_bonificacion() con su propia implementación.

9. Clase Desarrollador

```
class Desarrollador(Empleado):

    def calcular_bonificacion(self):
        return self.salario * 0.15
```

El desarrollador tiene exactamente el mismo método calcular_bonificacion(), pero hace algo diferente.

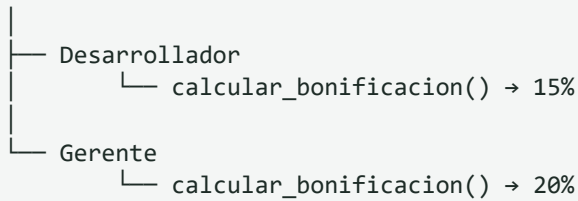
10. Clase Gerente

```
class Gerente(Empleado):

    def calcular_bonificacion(self):
        return self.salario * 0.20
```

Tenemos entonces:

```
Empleado
|
├── Administrativo
│   └── calcular_bonificacion() → 10%
```



Esto nos lleva directamente al polimorfismo.

11. Polimorfismo

Podemos hacer:

```
empleados = [
    Administrativo("Ana", "1001", 3000000),
    Desarrollador("Carlos", "1002", 5000000),
    Gerente("Laura", "1003", 8000000)
]
```

Y después:

```
for empleado in empleados:
    print(empleado.calcular_bonificacion())
```

Observa algo interesante: estamos llamando exactamente al mismo método, `empleado.calcular_bonificacion()`, pero Python ejecuta una implementación diferente dependiendo del objeto.

- Administrativo → 300.000
- Desarrollador → 750.000
- Gerente → 1.600.000

Eso es polimorfismo: un mismo método puede tener diferentes comportamientos dependiendo del objeto que lo invoque.

12. Encapsulamiento

Ahora vamos a mejorar nuestra clase. Supongamos que no queremos permitir que alguien coloque directamente un salario negativo. No queremos esto:

```
empleado.salario = -5000000
```

Podemos utilizar encapsulamiento. Una forma clásica es utilizar un atributo privado:

```
class Empleado(ABC):  
  
    def __init__(self, nombre, documento, salario):  
        self.nombre = nombre  
        self.documento = documento  
        self.__salario = salario
```

Observa `self.__salario`: los dos guiones bajos indican que estamos utilizando un atributo de acceso interno de la clase.

13. Getter con @property

Ahora necesitamos una forma controlada de consultar el salario:

```
@property  
def salario(self):  
    return self.__salario
```

Podemos hacer `print(empleado.salario)`, aunque internamente realmente estamos accediendo mediante la propiedad `@property`.

14. Setter

También podemos controlar cuándo alguien intenta modificar el salario:

```
@salario.setter  
def salario(self, nuevo_salario):  
  
    if nuevo_salario <= 0:  
        raise ValueError("El salario debe ser mayor que cero")  
  
    self.__salario = nuevo_salario
```

Ahora `empleado.salario = 6000000` es válido. Pero `empleado.salario = -100` produce:

```
ValueError: El salario debe ser mayor que cero
```

Esto es encapsulamiento: no dejamos que cualquiera modifique directamente el estado interno del objeto sin pasar por una validación.

15. __str__: imprimir un objeto

Aquí tenemos otra característica muy interesante. Supongamos:

```
empleado = Desarrollador(  
    "Carlos",  
    "123",  
    5000000  
)  
  
print(empleado)
```

¿Qué ocurriría si no tenemos __str__? Python normalmente mostraría algo parecido a:

```
<__main__.Desarrollador object at 0x000001A...>
```

Eso no es muy útil.

16. Sobrescribir __str__

Podemos hacer:

```
def __str__(self):  
    return f"{self.nombre} - Documento: {self.documento}"
```

Ahora print(empleado) produce:

```
Carlos - Documento: 123
```

Esto es una sobrescritura de un método especial. También podríamos hacer:

```
def __str__(self):  
    return (  
        f"Empleado: {self.nombre} | "  
        f"Documento: {self.documento} | "  
        f"Salario: ${self.salario:,.0f}"  
    )
```

Y obtendríamos:

```
Empleado: Carlos | Documento: 123 | Salario: $5,000,000
```

17. super()

Ahora quiero agregar un atributo adicional al desarrollador: lenguaje. Podríamos hacer:

```
class Desarrollador(Empleado):

    def __init__(self, nombre, documento, salario, lenguaje):
        super().__init__(nombre, documento, salario)
        self.lenguaje = lenguaje

    def calcular_bonificacion(self):
        return self.salario * 0.15
```

Aquí `super().__init__(nombre, documento, salario)` llama al constructor de la clase padre, es decir, `Empleado.__init__()`. Así evitamos repetir la asignación de nombre, documento y salario.

18. Ejemplo completo

Ahora podemos unir todo:

```
from abc import ABC, abstractmethod

class Empleado(ABC):

    def __init__(self, nombre, documento, salario):
        self.nombre = nombre
        self.documento = documento
        self.salario = salario

    @property
    def salario(self):
        return self.__salario

    @salario.setter
    def salario(self, nuevo_salario):

        if nuevo_salario <= 0:
            raise ValueError("El salario debe ser mayor que cero")

        self.__salario = nuevo_salario

    @abstractmethod
    def calcular_bonificacion(self):
        pass

    def mostrar_informacion(self):
```

```

        print(f"Nombre: {self.nombre}")
        print(f"Documento: {self.documento}")
        print(f"Salario: ${self.salario:,.0f}")

    def __str__(self):
        return (
            f"Empleado: {self.nombre} | "
            f"Documento: {self.documento} | "
            f"Salario: ${self.salario:,.0f}"
        )

class Administrativo(Empleado):

    def calcular_bonificacion(self):
        return self.salario * 0.10

class Desarrollador(Empleado):

    def __init__(self, nombre, documento, salario, lenguaje):
        super().__init__(nombre, documento, salario)
        self.lenguaje = lenguaje

    def calcular_bonificacion(self):
        return self.salario * 0.15

    def __str__(self):
        return (
            f"Desarrollador: {self.nombre} | "
            f"Lenguaje: {self.lenguaje} | "
            f"Salario: ${self.salario:,.0f}"
        )

class Gerente(Empleado):

    def calcular_bonificacion(self):
        return self.salario * 0.20

```

19. Programa principal

Ahora creamos:

```

def main():

    administrativo = Administrativo(
        "Ana",
        "1001",

```

```

        3000000
    )

    desarrollador = Desarrollador(
        "Carlos",
        "1002",
        5000000,
        "Python"
    )

    gerente = Gerente(
        "Laura",
        "1003",
        8000000
    )

    print(administrativo)
    print(desarrollador)
    print(gerente)

    print("\nBONIFICACIONES")

    print(
        f"{administrativo.nombre}: "
        f"${administrativo.calcular_bonificacion():,.0f}"
    )

    print(
        f"{desarrollador.nombre}: "
        f"${desarrollador.calcular_bonificacion():,.0f}"
    )

    print(
        f"{gerente.nombre}: "
        f"${gerente.calcular_bonificacion():,.0f}"
    )

if __name__ == "__main__":
    main()

```

20. ¿Qué imprime?

Tendríamos algo parecido a:

```

Empleado: Ana | Documento: 1001 | Salario: $3,000,000

Desarrollador: Carlos | Lenguaje: Python | Salario: $5,000,000

Empleado: Laura | Documento: 1003 | Salario: $8,000,000

```

BONIFICACIONES

Ana: \$300,000
 Carlos: \$750,000
 Laura: \$1,600,000

21. Ahora viene lo más interesante: polimorfismo real

Podemos mejorar el programa. En lugar de hacer:

```
print(administrativo.calcular_bonificacion())
print(desarrollador.calcular_bonificacion())
print(gerente.calcular_bonificacion())
```

hacemos:

```
empleados = [
    administrativo,
    desarrollador,
    gerente
]

for empleado in empleados:

    bonificacion = empleado.calcular_bonificacion()

    print(
        f"{empleado.nombre}: "
        f"${bonificacion:,.0f}"
    )
```

Y funciona. ¿Por qué? Porque todos los objetos son Empleado, pero cada uno implementa `calcular_bonificacion()` de manera diferente. Eso es justamente el polimorfismo.

22. Incluso podemos crear una función genérica

Podríamos hacer:

```
def mostrar_bonificacion(empleado):

    print(
        f"{empleado.nombre} recibe "
        f"${empleado.calcular_bonificacion():,.0f}"
```

```
)
```

Y:

```
mostrar_bonificacion(administrativo)
mostrar_bonificacion(desarrollador)
mostrar_bonificacion(gerente)
```

La función no necesita saber qué tipo de empleado recibió. Eso es una de las grandes ventajas de la POO.

23. ¿Qué pasa si intento crear Empleado?

Esto:

```
empleado = Empleado(
    "Pedro",
    "999",
    3000000
)
```

producirá un error, porque Empleado tiene un método `@abstractmethod` sin implementar, es decir, es una clase abstracta. Python no permite crear directamente un objeto de ella mientras tenga métodos abstractos sin implementar.

24. ¿Y qué pasa si creo una clase hija incorrecta?

Por ejemplo:

```
class Contratista(Empleado):
    pass
```

Y luego:

```
contratista = Contratista(
    "Pedro",
    "999",
    3000000
)
```

También tendremos un error, porque Contratista heredó `calcular_bonificacion()` pero no lo implementó. La clase abstracta obliga a las clases hijas a implementar ese comportamiento.

25. Ahora hagamos pruebas unitarias

Vamos a crear `test_empleados.py` y utilizaremos `import unittest`. Nuestro objetivo será comprobar automáticamente que nuestro código funciona.

26. Primera prueba

Queremos comprobar que un administrativo recibe el 10%.

```
import unittest

from empleados import Administrativo

class TestAdministrativo(unittest.TestCase):

    def test_bonificacion(self):

        empleado = Administrativo(
            "Ana",
            "1001",
            3000000
        )

        resultado = empleado.calcular_bonificacion()

        esperado = 300000

        self.assertEqual(resultado, esperado)

if __name__ == "__main__":
    unittest.main()
```

27. ¿Qué hace `assertEqual`?

```
self.assertEqual(resultado, esperado)
```

Significa: comprueba que ambos valores sean iguales. Estamos comprobando `resultado == esperado`. En este caso, `300000 == 300000`, entonces la prueba pasa.

28. Prueba del desarrollador

```
class TestDesarrollador(unittest.TestCase):

    def test_bonificacion(self):

        empleado = Desarrollador(
            "Carlos",
            "1002",
            5000000,
            "Python"
        )

        resultado = empleado.calcular_bonificacion()

        self.assertEqual(resultado, 750000)
```

Porque $5.000.000 \times 15\% = 750.000$.

29. Prueba del gerente

```
class TestGerente(unittest.TestCase):

    def test_bonificacion(self):

        empleado = Gerente(
            "Laura",
            "1003",
            8000000
        )

        resultado = empleado.calcular_bonificacion()

        self.assertEqual(resultado, 1600000)
```

30. Probar encapsulamiento

También podemos comprobar que el salario no pueda ser negativo.

```
class TestEmpleado(unittest.TestCase):

    def test_salario_no_puede_ser_negativo(self):

        empleado = Administrativo(
```

```

        "Ana",
        "1001",
        3000000
    )

    with self.assertRaises(ValueError):

        empleado.salario = -500000

```

Aquí, `assertRaises(ValueError)` significa: espero que este código produzca un `ValueError`. Por lo tanto, `empleado.salario = -500000` debe fallar, y eso es precisamente lo que queremos.

31. Probar `__str__`

También podemos probar la impresión del objeto.

```

class TestEmpleado(unittest.TestCase):

    def test_str(self):

        empleado = Administrativo(
            "Ana",
            "1001",
            3000000
        )

        resultado = str(empleado)

        esperado = (
            "Empleado: Ana | "
            "Documento: 1001 | "
            "Salario: $3,000,000"
        )

        self.assertEqual(resultado, esperado)

```

Estamos comprobando que `str(empleado)` devuelva exactamente lo que esperamos.

32. Archivo completo de pruebas

Podríamos dejar `test_empleados.py` así:

```

import unittest

from empleados import (

```

```

        Administrativo,
        Desarrollador,
        Gerente
    )

class TestAdministrativo(unittest.TestCase):

    def setUp(self):

        self.empleado = Administrativo(
            "Ana",
            "1001",
            3000000
        )

    def test_bonificacion(self):

        resultado = self.empleado.calcular_bonificacion()

        self.assertEqual(resultado, 300000)

    def test_nombre(self):

        self.assertEqual(
            self.empleado.nombre,
            "Ana"
        )

    def test_salario(self):

        self.assertEqual(
            self.empleado.salario,
            3000000
        )

class TestDesarrollador(unittest.TestCase):

    def setUp(self):

        self.empleado = Desarrollador(
            "Carlos",
            "1002",
            5000000,
            "Python"
        )

    def test_bonificacion(self):

        resultado = self.empleado.calcular_bonificacion()

```

```

        self.assertEqual(resultado, 750000)

    def test_lenguaje(self):

        self.assertEqual(
            self.empleado.lenguaje,
            "Python"
        )

class TestGerente(unittest.TestCase):

    def setUp(self):

        self.empleado = Gerente(
            "Laura",
            "1003",
            8000000
        )

    def test_bonificacion(self):

        resultado = self.empleado.calcular_bonificacion()

        self.assertEqual(resultado, 1600000)

class TestEncapsulamiento(unittest.TestCase):

    def test_salario_negativo(self):

        empleado = Administrativo(
            "Ana",
            "1001",
            3000000
        )

        with self.assertRaises(ValueError):

            empleado.salario = -100

class TestStr(unittest.TestCase):

    def test_str(self):

        empleado = Administrativo(
            "Ana",
            "1001",
            3000000
        )

```

```

        resultado = str(empleado)

        esperado = (
            "Empleado: Ana | "
            "Documento: 1001 | "
            "Salario: $3,000,000"
        )

        self.assertEqual(resultado, esperado)

if __name__ == "__main__":
    unittest.main()

```

33. Ejecutar las pruebas

Desde la terminal:

```
python -m unittest test_empleados.py
```

Podríamos obtener:

```

.....
-----
Ran 6 tests in 0.001s

OK

```

Los puntos (.....) significan que las pruebas pasaron.

34. ¿Qué ocurre si una prueba falla?

Supongamos que accidentalmente escribimos:

```
return self.salario * 0.20
```

en Desarrollador. La prueba espera 750000, pero recibirá 1000000. Entonces veremos algo parecido a:

```
FAIL: test_bonificacion
```

Y Python nos mostrará dónde ocurrió el problema. Esto es fundamental en desarrollo profesional.

35. Resumen de los 4 pilares

Ahora podemos relacionar todo:

Pilar / Concepto	En nuestro ejemplo
Abstracción	<code>Empleado(ABC)</code>
Abstracción	<code>@abstractmethod</code>
Herencia	<code>Administrativo(Empleado)</code>
Herencia	<code>Desarrollador(Empleado)</code>
Herencia	<code>Gerente(Empleado)</code>
Polimorfismo	<code>calcular_bonificacion()</code>
Encapsulamiento	<code>__salario</code>
Encapsulamiento	<code>@property</code>
Encapsulamiento	<code>@salario.setter</code>
Sobrescritura	<code>calcular_bonificacion()</code>
Método especial	<code>__str__()</code>
Constructor	<code>__init__()</code>
Herencia del constructor	<code>super().__init__()</code>
Pruebas	<code>unittest</code>

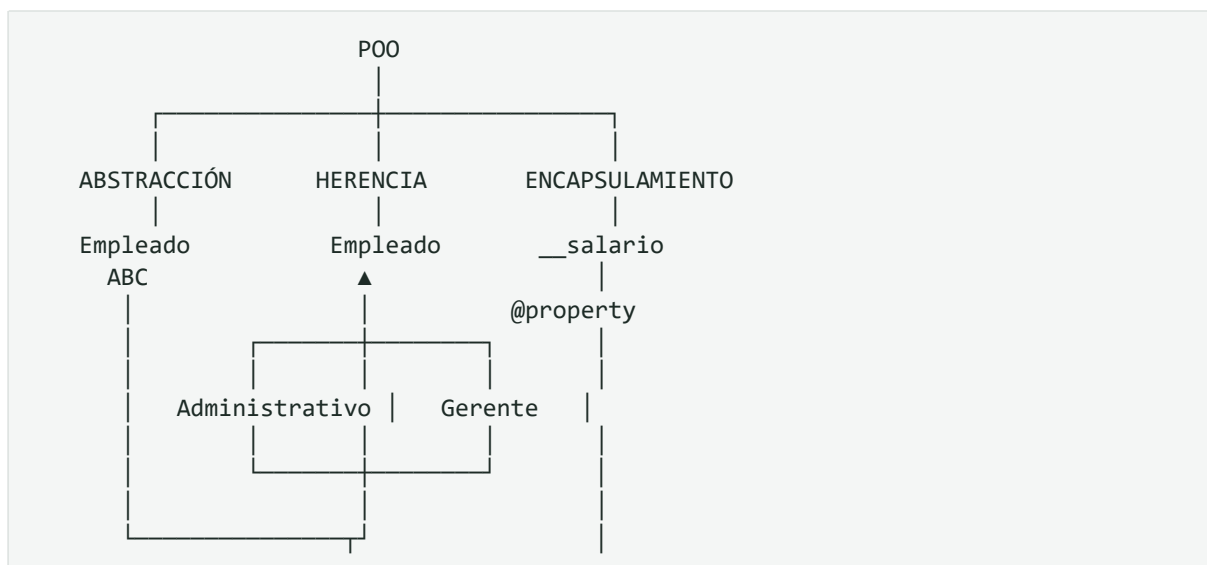
36. La idea que quiero que quede clara al estudiante

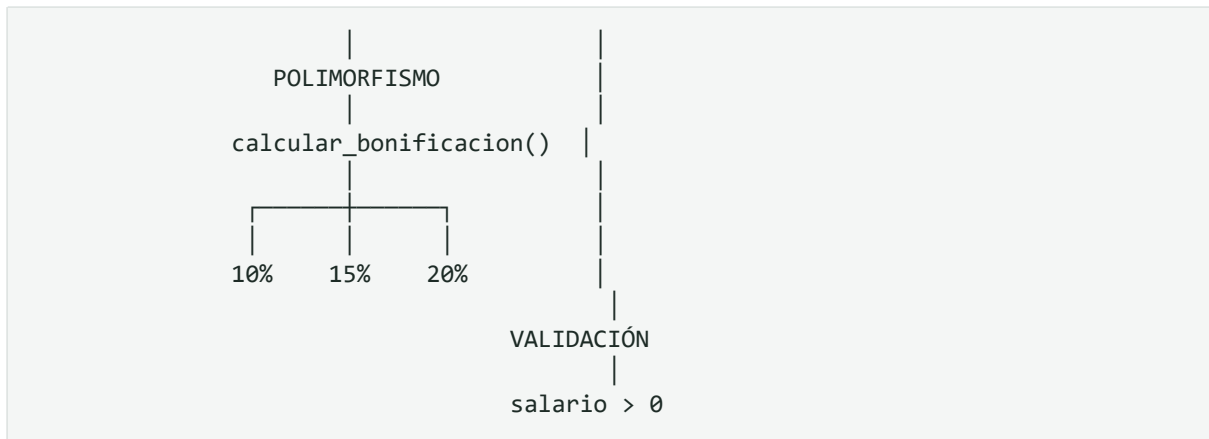
Una manera muy buena de explicarlo en clase es esta:

Concepto	Definición	Ejemplo
Clase	Es el molde.	<code>class Empleado:</code>
Objeto	Es una instancia del molde.	<code>ana = Administrativo(...)</code>
Atributo	Es un dato que pertenece al objeto.	<code>ana.nombre ana.salario</code>
Método	Es un comportamiento.	<code>ana.calcular_bonificacion()</code>
Herencia	Una clase aprovecha características de otra.	<code>class Desarrollador(Empleado):</code>
Abstracción	Definimos qué debe hacer una clase, no necesariamente cómo hacerlo.	<code>@abstractmethod def calcular_bonificacion(self):</code>
Polimorfismo	El mismo mensaje produce comportamientos diferentes.	<code>empleado.calcular_bonificacion()</code>
Encapsulamiento	Protegemos y controlamos los datos internos.	<code>self.__salario @property / @salario.setter</code>
Prueba unitaria	Comprobamos automáticamente que una parte del programa funcione bien.	<code>self.assertEqual(resultado, esperado)</code>

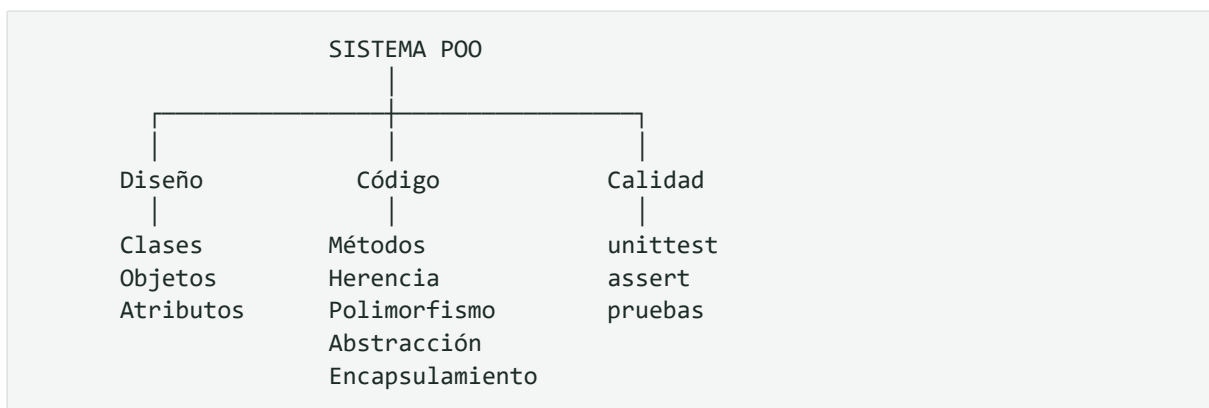
37. El mapa mental completo

La estructura conceptual sería:





Y finalmente:



Este es un muy buen ejemplo para enseñar POO porque los cuatro pilares no están metidos artificialmente: la abstracción define qué debe hacer un empleado, la herencia crea los tipos concretos, el polimorfismo permite calcular la bonificación sin preguntar qué tipo de empleado es, y el encapsulamiento protege el salario. Luego unittest permite demostrar que todo ese diseño puede probarse.